

CS471: Introduction to Artificial Intelligence

Assignment 3: Naive Bayes

Dax Taraleskof

Shane Dyrdaahl

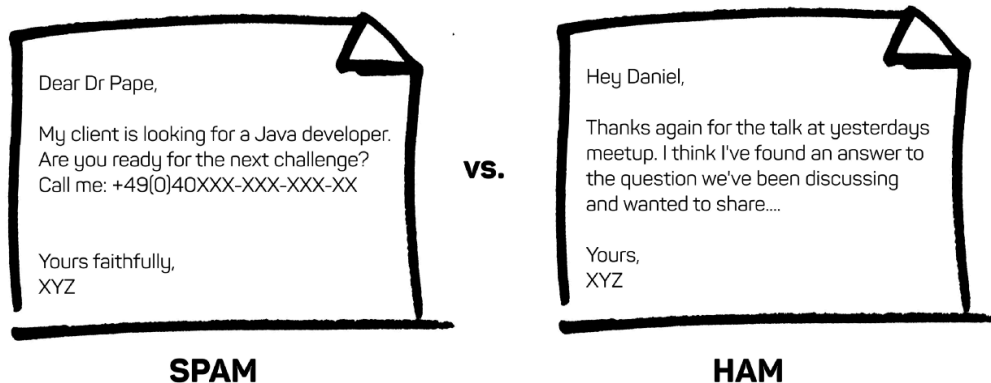
10/03/2024

In this assignment, you will implement the Naive Bayes classification method using Python.

You can either do this project individually or in a group of two. Use Google Colab for your code, plots, and comments. When you finish editing, re-run all the cells to make sure they work.

The Naive Bayes code must be your own. You are not allowed to use any machine learning libraries such as scikit-learn.

Goal is to classify a given message either a “spam” or a “ham”, using a Naive Bayes classifier.



For this assignment, you will be working with a Spam Collection dataset, consisting of text messages that have been collected for Spam research.

Dataset can be downloaded from here:

<https://drive.google.com/file/d/1MJzF39zIYnTkH4SfvW0TcTVXTrTAeDP-/view?usp=sharing>

The csv file contains one message per line with a total of 30 messages tagged either being ham (legitimate) or spam. Each line is composed of two columns: column 1 contains the label (ham or spam) and column 2 contains raw text.

Consider the first 20 samples as your training set and the rest 10 samples for your testing.

Tasks:

1. Load the dataset and split into training and testing sets (first 20 into training and the rest into testing) (1 point)
 2. Compute the prior probabilities: $P(\text{spam})$ and $P(\text{ham})$ (2 points)
 3. Compute the conditional probabilities $P(\text{sentence}/\text{spam})$ (2 points)
 4. Compute the posterior probabilities (probability of a sentence belonging to a spam or ham) (2 points)
 $P(\text{spam}/\text{sentence}) \propto P(\text{spam}) * P(\text{sentence}/\text{spam})$
Posterior $\propto$ prior * conditional
 $P(\text{ham}/\text{sentence}) \propto P(\text{ham}) * P(\text{sentence}/\text{ham})$
 5. For each sentence in the test set: (2 points)
Display the sentence
Print the posterior probability of a sentence belonging to spam or ham
Display the class (spam or ham)
 6. Report the test set accuracy (1 point)
Accuracy = no. of sentences correctly predicted by model / total sentences
-

Google colab link:

<https://colab.research.google.com/drive/13HwS4xcp4Q2ohXH1CHzjiKf7JXZEJ7m?usp=sharing>

We begin by importing the necessary modules into our project:

```
1 from typing import Dict, List, Tuple
2 from dataclasses import dataclass
3 from math import prod
4 from csv import reader
```

We use typing to help statically analyze our project in order to catch bugs before they appear at runtime. Using dataclasses, we define a 'Result' object, which holds all of the probability that are calculated for a given sentence. We use math.prod in order to help calculate the posterior probabilities for a given prior probability and dictionary of conditional probabilities. Finally, we use csv.reader to help transport the contents of the provided csv file into a list of 2-tuples, with position 1 being the classification of the sentence, and position 2 being the sentence itself.

We begin by defining our end goal, a dataclass that will hold the probabilities for each sentence:

```
6 @dataclass
7 class Result:
8     sentence: str
9     prior: float
10    conditional: Dict[str, float]
11    posterior: float
```

We continue by utilizing the aforementioned csv module, specifically the reader method, which we use to define a function that reads the data from SpamDetection.csv into a list of 2-tuples:

```

13 def listify_csv(file_name: str) -> List[Tuple[str, str]]:
14     """
15     Attempts to open csv file with two columns: class,data
16     and then returns it as a List of Tuples for each row.
17     """
18     with open(file_name) as f:
19         csv_reader = reader(f)
20
21         # skip the header objects
22         next(csv_reader)
23
24         contents = []
25         for row in csv_reader:
26             contents.append(row)
27
28     return contents
29

```

This is used later on in `__main__` to satisfy the requirements of task 1:

```

167 if __name__ == "__main__":
168     print("\nProblem from class:")
169     test_case()
170
171     training_amount = 20
172     contents = listify_csv("SpamDetection.csv")
173     # Task 1. Split into testing set (training set is done in calculate_probabilities as the argument 'split')
174     testing_set = contents[training_amount:]
175     print("\nTesting Set:", testing_set, len(testing_set))

```

With the variable 'training_amount' being the required split in task 1, which by using slice syntax, we split into another list, called 'testing_set', which will be used later. For each row in the testing list, we iterate through it and call our main function, `calculate_probabilities()`, which calculates the prior, conditional, and posterior probabilities that have been trained on the data set (contents) for each of the sentences in the testing_set:

```

177 print("\nTest Set from Assignment:")
178 results = []
179 for classification, sentence in testing_set:
180     spam_result, ham_result = calculate_probabilities(
181         sentence=sentence,
182         split=training_amount,
183         data=contents,
184     )

```

The following is the code for `calculate_probabilities()`, which is responsible for tasks 1-4 and returns a tuple of the 'Result' dataclass for each of the sentences in the testing_set:

```
104 def calculate_probabilities(  
105     sentence: str,  
106     split: int,  
107     data: List[Tuple[str, str]],  
108 ) -> Tuple[Result, Result]:  
109     """  
110     Calculates the prior, conditional, and posterior probabilities for the  
111     given sentence and return a tuple of Results for ham and spam.  
112     """  
113     # Task 1. Load the dataset and split into training and testing sets (first 20 into training and the rest into testing)  
114     spam_training, ham_training = categorize(data, split) # split=20  
115     # Union of the set of keys from both training dictionaries. This filters any duplicates and returns the unique word count.  
116     unique_word_count = len(set(spam_training.keys() | ham_training.keys()))  
117     # Task 2. Calculate the prior probabilities: P(spam) and P(ham) (2 points)  
118     p_spam_prior, p_ham_prior = probability_prior(data, split)  
119     # Task 3. Calculate conditional probabilities: P(sentence|ham) and P(sentence|spam) (2 points)  
120     p_ham_conditional = probability_conditional(sentence, unique_word_count, ham_training)  
121     p_spam_conditional = probability_conditional(sentence, unique_word_count, spam_training)  
122     # Task 4. Compute the posterior probabilities: P(sentence|ham) and P(sentence|spam) (2 points)  
123     # The resulting float value is the posterior probability P(ham|sentence) or P(spam|sentence) for the given sentence.  
124     p_ham_posterior = probability_posterior(p_ham_prior, p_ham_conditional)  
125     p_spam_posterior = probability_posterior(p_spam_prior, p_spam_conditional)  
126     # Returns all of the computed probabilities as a python dataclass called 'Result', which is defined at the top of this file.  
127     return Result(sentence, p_spam_prior, p_spam_conditional, p_spam_posterior), Result(sentence, p_ham_prior, p_ham_conditional, p_ham_posterior)
```

Tasks

1. We will now be discussing each of the functions called in the `calculate_probabilities()` function, starting with `categorize()`, which accepts the data as a list, as well as an integer that is the cutoff point for our training set. This function accepts the original list of data but only reads up to the designated 'training_amount'.

```
30 def categorize(  
31     data: List[Tuple[str, str]],  
32     training_amount: int  
33 ) -> Tuple[Dict[str, int], Dict[str, int]]:  
34     """  
35     Returns a Dictionary of every word in the data list after it has been  
36     classified as Ham or Spam.  
37     """  
38     spam, ham = {}, {}  
39     for idx in range(training_amount):  
40         row = data[idx]  
41         words = row[1].rstrip().split(" ")  
42         for w in words:  
43             w = w.lower().strip("")  
44             if row[0] == "ham":  
45                 if w not in ham:  
46                     ham[w] = 1  
47                 else:  
48                     ham[w] += 1  
49             elif row[0] == "spam":  
50                 if w not in spam:  
51                     spam[w] = 1  
52                 else:  
53                     spam[w] += 1  
54  
55     return spam, ham
```

2. We define the `probability_prior()` function, which calculates the prior probabilities for both spam and ham based on the original data list up to `data_num`.

```
57 def probability_prior(  
58     data: List[Tuple[str, str]],  
59     data_num: int  
60 ) -> Tuple[float, float]:  
61     """  
62     Compute the prior probability for {data_num} amount of rows and returns  
63     two float values, the first being for SPAM and the second being for HAM.  
64     """  
65     total, spam_total, ham_total = 0, 0, 0  
66     for row in data:  
67         if data_num - total == 0: break  
68         if row[0] == "spam":  
69             spam_total += 1  
70         if row[0] == "ham":  
71             ham_total += 1  
72         total += 1  
73  
74     return spam_total / total, ham_total / total
```

3. We define the `probability_conditional()` function, which calculates the conditional probabilities for the given sentence. For each occurrence of the word, we calculate the $P(\text{sentence} \mid \text{spam})$ or $P(\text{sentence} \mid \text{ham})$ while ignoring spaces. If a word does not appear in any of the occurrences, then we apply laplace smoothing to the conditional probability to prevent a 0 probability.

```
76 def probability_conditional(  
77     sentence: str,  
78     unique_words: int,  
79     occurrence_per_word: Dict[str, int],  
80 ) -> Dict[str, float]:  
81     """Calculates the conditional probability for each word in a sentence and applies Laplace Smoothing."""  
82     total = sum(occurrence_per_word.values())  
83     laplace_smoothing = total + unique_words  
84  
85     conditional = {}  
86     words = sentence.split(" ")  
87     for w in words:  
88         if w.isspace():  
89             continue  
90         w = w.lower()  
91         if w not in occurrence_per_word:  
92             conditional[w] = 1 / laplace_smoothing  
93         if w in occurrence_per_word:  
94             conditional[w] = (occurrence_per_word[w] + 1) / laplace_smoothing  
95  
96     return conditional
```

4. We calculate the posterior probability given a dictionary of conditional probabilities ($P(\text{sentence}|\text{spam})$ or $P(\text{sentence}|\text{ham})$) and the prior probability of that classification ($P(\text{ham})$ or $P(\text{spam})$), which in this case is the posterior formula: $\text{posterior} = \text{prior} * \text{conditional}$

```
98 def probability_posterior(  
99     prior_probability: float,  
100     conditional_probability: Dict[str, float],  
101 ) -> float:  
102     """Calculates the posterior probabilities given a set of conditional and prior values for both spam and ham."""  
103     return prod(conditional_probability.values()) * prior_probability
```

This concludes the walkthrough of the calculate_probabilities() function, which returns a Result of each of the aforementioned probabilities by using each of the functions discussed.

5. We print the currently tested sentence, its posterior probabilities, and how the model classified it:

```
194 # Task 5. For each sentence in the test set: (2 points)  
195 # Display the sentence  
196 # Print the posterior probability of a sentence belonging to spam or ham  
197 # Display the class (spam or ham)  
198 actual_class = "spam" if spam_result.posterior > ham_result.posterior else "ham"  
199 print(f"\nSentence: '{sentence}'")  
200 print(f"Posterior Probabilities: SPAM={spam_result.posterior} | HAM={ham_result.posterior}")  
201 print(f"Classification: ACTUAL={actual_class.upper()} | EXPECTED={classification.upper()}")
```

6. We create a function to calculate the accuracy of the model for both the testing and training sets by comparing the 'ACTUAL' results to the 'EXPECTED' result from the original test set and original training set:

```
130 def calculate_accuracy(  
131     testing_set: List[Tuple[str, str]],  
132     results: List[Tuple[Result, Result]]  
133 ) -> float:  
134     """  
135     Compares actual results from the results list to the original testing_set and calculates  
136     the accuracy of the Naive Bayes model.  
137     """  
138     correct_predictions = 0  
139     for idx, (classification, sentence) in enumerate(testing_set):  
140         spam_result, ham_result = results[idx]  
141         predicted_label = "spam" if spam_result.posterior > ham_result.posterior else "ham"  
142         if predicted_label == classification:  
143             correct_predictions += 1  
144     accuracy = correct_predictions / len(testing_set)  
145     return accuracy
```

This function is called in `__main__`:

```
227 # Report the train and test set accuracy (1 point)
228 # Accuracy = no. of sentences correctly predicted by model / total sentences
229 accuracy_of_model_training = calculate_accuracy(contents[:training_amount], training_set_results)
230 print(f"\nTraining set accuracy : {accuracy_of_model_training * 100:.2f}%")
231
232 accuracy_of_model_test = calculate_accuracy(testing_set, testing_set_results)
233 print(f"\nTesting set accuracy : {accuracy_of_model_test * 100:.2f}%")
```

Test Set:

Test Set from Assignment:

Sentence: 'Tell where you reached'
Posterior Probabilities: SPAM=2.4500995093215107e-10 | HAM= 2.6086421175275796e-09
Classification: ACTUAL=HAM | EXPECTED=HAM

Sentence: 'Your gonna have to pick up a burger for yourself on your way home'
Posterior Probabilities: SPAM=5.993484267252724e-30 | HAM= 2.2828236353215773e-29
Classification: ACTUAL=HAM | EXPECTED=HAM

Sentence: 'As a valued customer I am pleased to advise you that for your recent review you are awarded a Bonus Prize'
Posterior Probabilities: SPAM=4.973710939159712e-43 | HAM= 2.796284443819346e-42
Classification: ACTUAL=HAM | EXPECTED=SPAM

Sentence: 'Urgent you are awarded a complimentary trip to EuroDisinc To claim text immediately'
Posterior Probabilities: SPAM=5.3263238471036776e-27 | HAM= 1.3754012402812503e-27
Classification: ACTUAL=SPAM | EXPECTED=SPAM

Sentence: 'Finished class where are you'
Posterior Probabilities: SPAM=1.512407104519451e-12 | HAM= 1.082424115156672e-11
Classification: ACTUAL=HAM | EXPECTED=HAM

Sentence: 'where are you how did you perform '
Posterior Probabilities: SPAM=1.440717025338602e-17 | HAM= 2.981833274652081e-15
Classification: ACTUAL=HAM | EXPECTED=HAM

Sentence: 'you can call me now'
Posterior Probabilities: SPAM=1.5880274597454233e-11 | HAM= 4.329696460626688e-11
Classification: ACTUAL=HAM | EXPECTED=HAM

Sentence: 'I am waiting Call me once you are free'
Posterior Probabilities: SPAM=4.80348802458191e-21 | HAM= 3.080353239082055e-19
Classification: ACTUAL=HAM | EXPECTED=HAM

Sentence: 'I am on the way to homei'
Posterior Probabilities: SPAM=9.244600912589363e-17 | HAM= 2.2363749559890604e-15
Classification: ACTUAL=HAM | EXPECTED=HAM

Sentence: 'Please call our customer service representative between 10am-9pm as you have WON a guaranteed cash prize'
Posterior Probabilities: SPAM=8.304560373630298e-36 | HAM= 4.077192731701523e-37
Classification: ACTUAL=SPAM | EXPECTED=SPAM

Training set accuracy : 100.00%

Testing set accuracy : 90.00%